Name-Rohan Lalchandani | D20A | Roll 31

Experiment: 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Theory:

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy smart contracts and decentralized applications (DApps). Unlike public blockchains such as the Ethereum mainnet, a Private Ethereum Blockchain is restricted to specific participants, making it ideal for development, testing, and enterprise applications where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through JSON-RPC, command-line, or JavaScript consoles.

Setting up a **Private Ethereum Network** involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

1. **Choosing a Network ID:**
2. Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
3. **Choosing a Consensus Algorithm:**
4. Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation. 5.

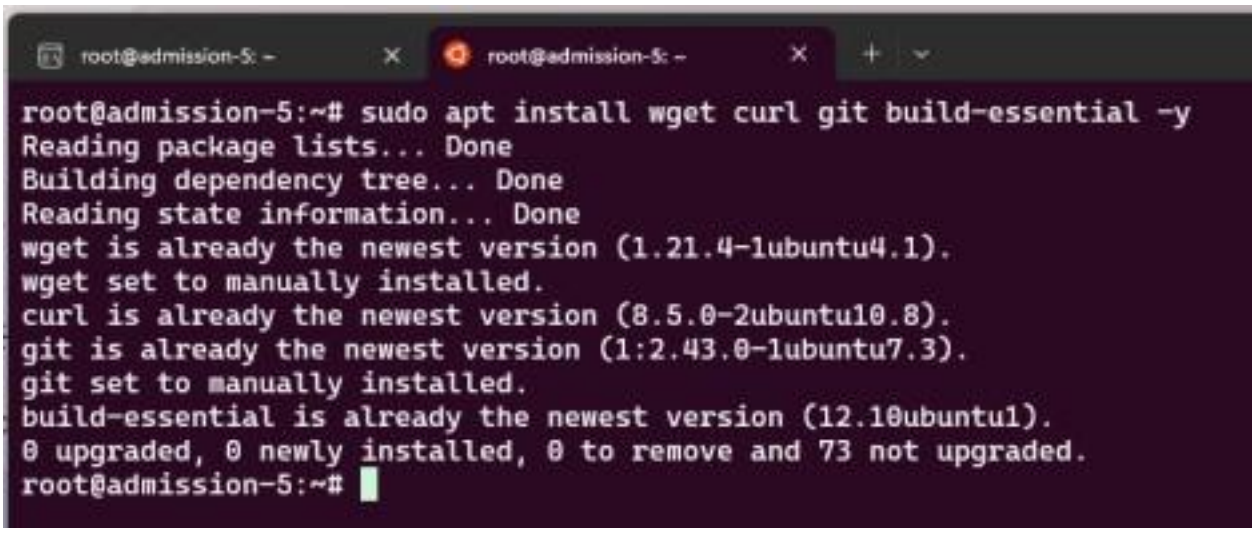
Creating a Genesis Block:

6. The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
7. **Initializing the Geth Database:**
8. The geth init command initializes the blockchain using the **genesis.json** file, preparing the data directory for node operation.
9. **Setting up Networking:**
10. Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
11. **Running the Member Nodes:**
12. Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
13. **Running a Signer (in Clique):**

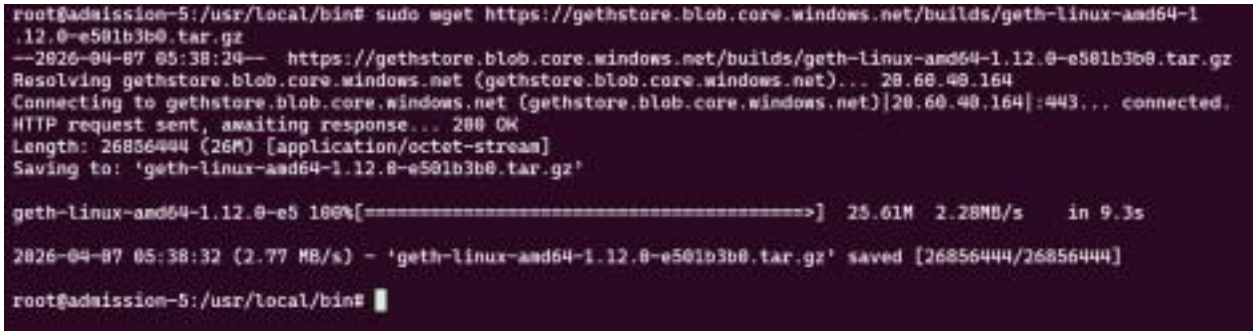
14. In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Implementation:

1. First install all the necessary dependencies like curl, git, build-essential. sudo apt install wget curl git build-essential -y



2. Install geth using wget cd /usr/local/bin && sudo wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz



3. Extract the tar file and move the geth into /usr/local/bin/geth path & check the geth version sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth geth version

```
root@admission-5:/usr/local/bin# sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz
sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth
geth version
geth-linux-amd64-1.12.0-e501b3b0/
geth-linux-amd64-1.12.0-e501b3b0/COPYING
geth-linux-amd64-1.12.0-e501b3b0/geth
Geth
Version: 1.12.0-stable
Git Commit: e501b3b05db8e169f67dc78b7b59bc352b3c638d
Git Commit Date: 20230525
Architecture: amd64
Go Version: go1.20.3
Operating System: linux
GOPATH=
GOROOT=
root@admission-5:/usr/local/bin#
```

4. Create a folder named private_ethereum_setup and create another 2 directories naming node1 and node2 individually inside private_ethereum_setup directory. mkdir private_ethereum_setup cd private_ethereum_setup && mkdir node1 node2

```
root@admission-5:/usr/local/bin# mkdir private_ethereum_setup
root@admission-5:/usr/local/bin# cd private_ethereum_setup && mkdir node1 node2
root@admission-5:/usr/local/bin/private_ethereum_setup# ls
node1  node2
root@admission-5:/usr/local/bin/private_ethereum_setup#
```

5. Create account for node1 geth --datadir "node1" account new

```
root@admission-5:/usr/local/bin/private_ethereum_setup# geth --datadir "node1" account new
INFO [04-07|05:42:29.002] Maximum peer count                       ETH=50 LES=0 total=50
INFO [04-07|05:42:29.003] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0xB39388323dCD82eeb28F73B3448424c008FE7150
Path of the secret key file: node1/keystore/UTC--2026-04-07T05-42-42.824513266Z--b39388323dcd82eeb28f73b3448424c008fe7150

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

root@admission-5:/usr/local/bin/private_ethereum_setup#
```

6. Do the same for node2 geth --datadir "node2" account new

```
root@admission-5:/usr/local/bin/private_ethereum_setup# geth --datadir "node2" account new
INFO [04-07|05:43:28.007] Maximum peer count                       ETH=50 LES=0 total=50
INFO [04-07|05:43:28.007] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0x126F82349979c6b3d2a52eE032B7ea11880D2dC9
Path of the secret key file: node2/keystore/UTC--2026-04-07T05-43-35.386933461Z--126f82349979c6b3d2a52ee032b7ea11880d2dc9

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

root@admission-5:/usr/local/bin/private_ethereum_setup#
```

7. Create a file called genesis.json in the same directory and copy paste these contents

```
{
 "config": {
 "chainId": 12345,
 "homesteadBlock": 0,
 "eip150Block": 0,
 "eip155Block": 0,
 "eip158Block": 0,
 "byzantiumBlock": 0,
 "constantinopleBlock": 0,
 "petersburgBlock": 0,
 "istanbulBlock": 0,
 "berlinBlock": 0,
 "londonBlock": 0,
 "clique": {
 "period": 5,
 "epoch": 30000
 }
 },
 "difficulty": "1",
 "gasLimit": "8000000",
 "extradata":
"0x0000000000000000000000000000000000000000000000000000000000
00000000c99cf64ef49425bf0e814c9d51364281a61747d200000000000
0000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000
00000",
 "alloc": {
 "0xB39388323dCD82eeb28F73B3448424c008FE7150": {
 "balance": "100000000000000000000"
 },
 "0x126F82349979c6b3d2a52eE032B7ea11880D2dC9": {
 "balance": "100000000000000000000"
 }
 }
}
```

8. Cat genesis.json

```
root@admission-5:/usr/local/bin/private_ethereum_setup# cat genesis.json
{
 "config": {
 "chainId": 12345,
 "homesteadBlock": 0,
 "eip150Block": 0,
 "eip155Block": 0,
 "eip158Block": 0,
 "byzantiumBlock": 0,
 "constantinopleBlock": 0,
 "petersburgBlock": 0,
 "istanbulBlock": 0,
 "berlinBlock": 0,
 "londonBlock": 0,
 "clique": {
 "period": 5,
 "epoch": 30000
 }
 },
 "difficulty": "1",
 "gasLimit": "8000000",
 "extradata":
"0x0000000000000000000000000000000000000000000000000000000000000000
00000000c99cf64ef49425bf0e814c9d51364281a61747d200000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
00000",
 "alloc": {
 "0xB393BB323dCD82eeb28F73B3448424c008FE7150": {
 "balance": "100000000000000000000"
 },
 "0x126F82349979c6b3d2a52eE032B7ea11880D2dC9": {
 "balance": "100000000000000000000"
```

9. Initializing geth for node1 using the genesis.json file geth init --datadir node1 genesis.json

```
]root@admission-5:/usr/local/bin/private_ethereum_setup# geth init --datadir node1 genesis.json
INFO [04-07|05:58:25.537] Maximum peer count                       ETH=50 LES=0 total=50
INFO [04-07|05:58:25.538] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-07|05:58:25.543] Set global gas cap                       cap=50,000,000
INFO [04-07|05:58:25.544] Initializing the KZG library             backend=gokzg
INFO [04-07|05:58:25.594] Defaulting to pebble as the backing database
INFO [04-07|05:58:25.595] Allocated cache and file handles         database=/usr/local/bin/private_ethereum_setup/node1/geth/chaindat
a cache=16.00MiB handles=16
INFO [04-07|05:58:25.618] Opened ancient database                  database=/usr/local/bin/private_ethereum_setup/node1/geth/chaindat
a/ancient/chain readonly=false
INFO [04-07|05:58:25.619] Writing custom genesis block
INFO [04-07|05:58:25.619] Persisted trie from memory database      nodes=3 size=411.00B time="9.642µs" gcnodes=0 gcsize=0.00B gctime=
0s livenodes=0 livesize=0.00B
INFO [04-07|05:58:25.623] Successfully wrote genesis state         database=chaindata hash=8c5dd7..24caed
INFO [04-07|05:58:25.623] Defaulting to pebble as the backing database
INFO [04-07|05:58:25.623] Allocated cache and file handles         database=/usr/local/bin/private_ethereum_setup/node1/geth/lightcha
indata cache=16.00MiB handles=16
INFO [04-07|05:58:25.640] Opened ancient database                  database=/usr/local/bin/private_ethereum_setup/node1/geth/lightcha
indata/ancient/chain readonly=false
INFO [04-07|05:58:25.641] Writing custom genesis block
INFO [04-07|05:58:25.641] Persisted trie from memory database      nodes=3 size=411.00B time="10.121µs" gcnodes=0 gcsize=0.00B gctime
=0s livenodes=0 livesize=0.00B
INFO [04-07|05:58:25.644] Successfully wrote genesis state         database=lightchaindata hash=8c5dd7..24caed
root@admission-5:/usr/local/bin/private_ethereum_setup#
```

10. Do the same for node2 geth init --datadir node2 genesis.json

```
root@admission-5:/usr/local/bin/private_ethereum_setup# geth init --datadir node2 genesis.json
INFO [04-07|05:59:40.725] Maximum peer count                       ETH=50 LES=0 total=50
INFO [04-07|05:59:40.726] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-07|05:59:40.729] Set global gas cap                       cap=50,000,000
INFO [04-07|05:59:40.729] Initializing the KZG library             backend=gokzg
INFO [04-07|05:59:40.760] Defaulting to pebble as the backing database
INFO [04-07|05:59:40.760] Allocated cache and file handles         database=/usr/local/bin/private_ethereum_setup/node2/geth/chaindat
a cache=16.00MiB handles=16
INFO [04-07|05:59:40.780] Opened ancient database                  database=/usr/local/bin/private_ethereum_setup/node2/geth/chaindat
a/ancient/chain readonly=false
INFO [04-07|05:59:40.780] Writing custom genesis block
INFO [04-07|05:59:40.780] Persisted trie from memory database      nodes=3 size=411.00B time="10.186µs" gcnodes=0 gcsize=0.00B gctime
=0s livenodes=0 livesize=0.00B
INFO [04-07|05:59:40.784] Successfully wrote genesis state         database=chaindata hash=0c5dd7..24caed
INFO [04-07|05:59:40.784] Defaulting to pebble as the backing database
INFO [04-07|05:59:40.784] Allocated cache and file handles         database=/usr/local/bin/private_ethereum_setup/node2/geth/lightcha
indata cache=16.00MiB handles=16
INFO [04-07|05:59:40.805] Opened ancient database                  database=/usr/local/bin/private_ethereum_setup/node2/geth/lightcha
indata/ancient/chain readonly=false
INFO [04-07|05:59:40.805] Writing custom genesis block
INFO [04-07|05:59:40.805] Persisted trie from memory database      nodes=3 size=411.00B time="10.09µs"  gcnodes=0 gcsize=0.00B gctime
=0s livenodes=0 livesize=0.00B
INFO [04-07|05:59:40.808] Successfully wrote genesis state         database=lightchaindata hash=0c5dd7..24caed
root@admission-5:/usr/local/bin/private_ethereum_setup#
```

11. Now we will install bootnode for configuring both the nodes. sudo add-apt-repository -y ppa:ethereum/ethereum sudo apt-get update sudo apt-get install bootnode

```
root@admission-5:/usr/local/bin/private_ethereum_setup# sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt-get update
sudo apt-get install bootnode
Repository: 'Types: deb
URIs: https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu/
Suites: noble
Components: main
'
More info: https://launchpad.net/~ethereum/+archive/ubuntu/ethereum
Adding repository.
Hit:1 https://deb.nodesource.com/node_18.x nodistro InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:6 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble InRelease [18.1 kB]
Get:7 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble/main amd64 Packages [2576 B]
Get:8 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble/main Translation-en [772 B]
Fetched 21.4 kB in 1s (18.3 kB/s)
Reading package lists... Done
Hit:1 https://deb.nodesource.com/node_18.x nodistro InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu noble InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  bootnode
0 upgraded, 1 newly installed, 0 to remove and 73 not upgraded.
```

12. Create a key for the bootnode and save it to boot.key in the folder bootnode -genkey boot.key

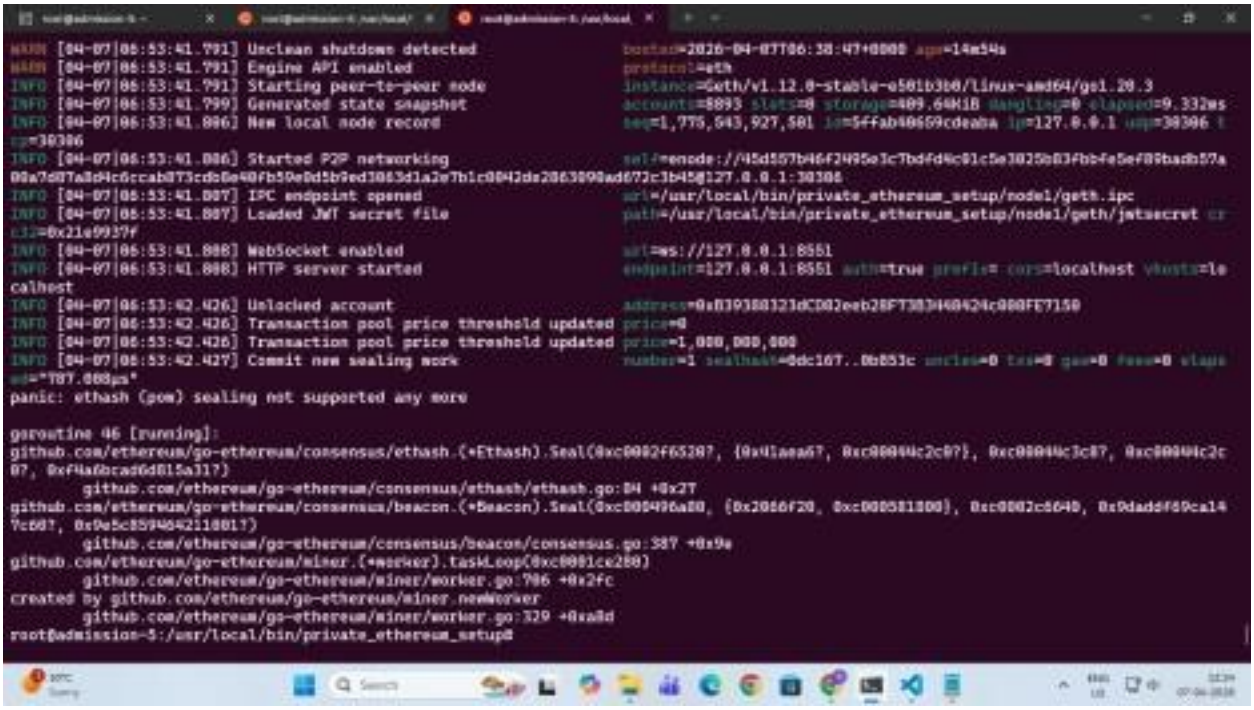
```
root@admission-5:/usr/local/bin/private_ethereum_setup# bootnode -genkey boot.key
root@admission-5:/usr/local/bin/private_ethereum_setup# ls
boot.key  genesis.json  node1  node2
root@admission-5:/usr/local/bin/private_ethereum_setup#
```

13. Copy this command and copy the enode://…. This will be required later bootnode -nodekey boot.key -verbosity 9 -addr :30305

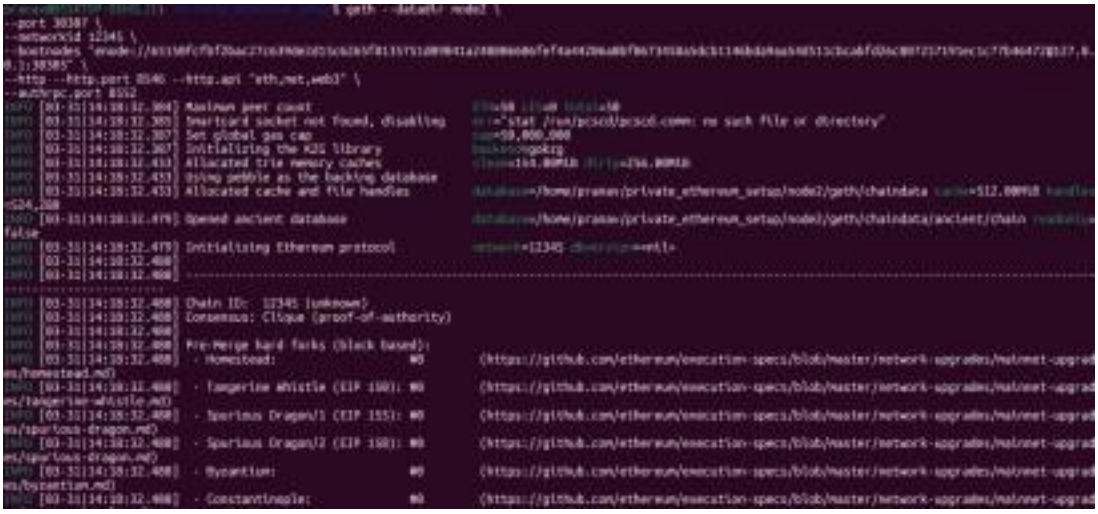
```
root@admission-5:/usr/local/bin/private_ethereum_setup# bootnode -nodekey boot.key -verbosity 9 -addr :30305
enode://7ff7a90d47bc73762802a4f924fdcb403af4bff08ded3c3b0b5c2efeb9f76553859f6edf9abdf20619c04c9408fc7ed606e097a26d4519d50f5afcc063e02
120@127.0.0.1:0?discport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [04-07|06:03:30.800] New local node record                    seq=1,775,541,810,798 id=f572303c2cd66d79 ip="invalid IP" udp=0 tc
p=0
```

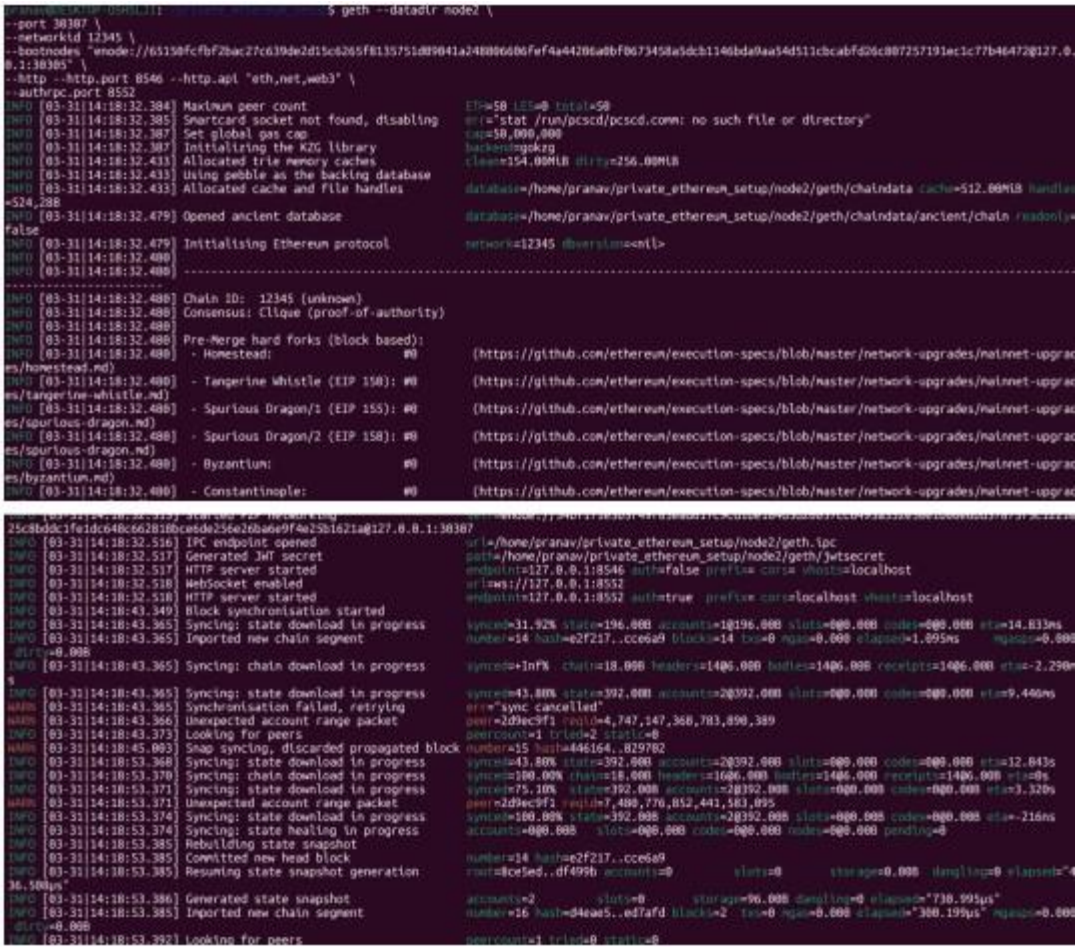
14. Now on another terminal we will start mining node1 geth --datadir node1 \
--port 30306 \
--bootnodes "enode://7ff7a90d47bc73762802a4f924fdcb403af4bff08ded3c3b0b5c2efeb9f76553859f6edf9abdf20619c04c9408fc7ed606e097a26d4519d50f5afcc063e02120@127.0.0.1:30305" \
--networkid 123454321 \
--unlock 0xB39388323dCD82eeb28F73B3448424c008FE7150 \
--password node1/password.txt \
--authrpc.port 8551 \
--miner.etherbase 0xB39388323dCD82eeb28F73B3448424c008FE7150 \
--mine

```
root@admission-5:/usr/local/bin/private_ethereum_setup# rm -rf /usr/local/bin/private_ethereum_setup/node1/geth
root@admission-5:/usr/local/bin/private_ethereum_setup# geth --datadir node1 --port 30306 --bootnodes "enode://7ff7a90d47bc73762802a4
f924fdcb403af4bff08ded3c3b0b5c2efeb9f76553859f6edf9abdf20619c04c9408fc7ed606e097a26d4519d50f5afcc063e02120@127.0.0.1:30305" --network
id 123454321 --unlock 0xB39388323dCD82eeb28F73B3448424c008FE7150 --password node1/password.txt --authrpc.port 8551 --miner.etherbase
0xB39388323dCD82eeb28F73B3448424c008FE7150 --mine
INFO [04-07|06:38:47.164] Maximum peer count                       ETH=50 LES=0 total=50
INFO [04-07|06:38:47.166] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-07|06:38:47.168] Set global gas cap                       cap=50,000,000
INFO [04-07|06:38:47.169] Initializing the KZG library             backend=gokzg
INFO [04-07|06:38:47.199] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [04-07|06:38:47.199] Defaulting to pebble as the backing database
INFO [04-07|06:38:47.199] Allocated cache and file handles         database=/usr/local/bin/private_ethereum_setup/node1/geth/chaindat
a cache=512.00MiB handles=524,288
INFO [04-07|06:38:47.225] Opened ancient database                  database=/usr/local/bin/private_ethereum_setup/node1/geth/chaindat
a/ancient/chain readonly=false
INFO [04-07|06:38:47.225] Initialising Ethereum protocol           network=123,454,321 dbversion=<nil>
INFO [04-07|06:38:47.226] Writing default main-net genesis block
INFO [04-07|06:38:47.461] Persisted trie from memory database      nodes=12356 size=1.79MiB time=48.92926ms gcnodes=0 gcsize=0.00B gc
time=0s livenodes=0 livesize=0.00B
INFO [04-07|06:38:47.476]
INFO [04-07|06:38:47.476] ----------------------------------------------------------------------------------------------------
----------------------------------------
INFO [04-07|06:38:47.476] Chain ID:  1 (mainnet)
INFO [04-07|06:38:47.476] Consensus: Beacon (proof-of-stake), merged from Ethash (proof-of-work)
INFO [04-07|06:38:47.476]
INFO [04-07|06:38:47.476] Pre-Merge hard forks (block based):
INFO [04-07|06:38:47.476]  - Homestead:                    #1150000  (https://github.com/ethereum/execution-specs/blob/master/network-
upgrades/mainnet-upgrades/homestead.md)
INFO [04-07|06:38:47.476]  - DAO Fork:                     #1920000  (https://github.com/ethereum/execution-specs/blob/master/network-
upgrades/mainnet-upgrades/dao-fork.md)
INFO [04-07|06:38:47.476]  - Tangerine Whistle (EIP 150): #2463000  (https://github.com/ethereum/execution-specs/blob/master/network-
upgrades/mainnet-upgrades/tangerine-whistle.md)
```



15. Now do same for node2





16. Now we will interact with the blockchain using javascript console geth attach node1/geth.ipc

17. Interacting with the blockchain

```
> eth.getBalance(eth.accounts[0])
> web3.fromWei(eth.getBalance("0x563a5a670260417b163ad1696fcbfa6849906d06"), "ether")

> eth.getBlock(eth.blockNumber)
{
  baseFeePerGas: 10300,
  difficulty: 1,
  extraData: "0xd883010c0084676574688867 6f312e32382e33856c696e75780000000000000084131cba2f966b7937d8d78eb13d85086a2a85d6e26959dfb0e771f0987e27f32
0e09094920833 2f2cdc5e7aecbf80ad7392c1d7541ec88859cec7cfb4c14aca01",
  gasLimit: 4300000,
  gasUsed: 0,
  hash: "0x0fdff9b159410966 1ff4afdf42b06c14187de1d8c4ecb279a42e41fa1a240a5b",
  logsBloom: "0x0000000000000
```

```
> eth.getBlock(355)
{
  baseFeePerGas: 7,
  difficulty: 2,
  extraData: "0xd883010c0884676574688867 6f312e32382e33856c696e7578000000000000054e681ad4052a191aaff4b0e0424dc223f194b9c05561
5bfb86960bc387b199878645b27b2f6cd8e758771fd09e0d1c1de0c63a7ebf214219ae29612c057212600",
  gasLimit: 11312359,
  gasUsed: 0,
  hash: "0x687ab31e5c2c7d347d56020734532d38028c0bc6907f622033620759711fbc00",
  logsBloom: "0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000",
  miner: "0x0000000000000000000000000000000000000000",
  mixHash: "0x0000000000000000000000000000000000000000000000000000000000000000",
  nonce: "0x0000000000000000",
  number: 355,
  parentHash: "0xe0da19a2c7dbd84c2356a7d5430f06d7b8759a91647a84fd813277b5af2dd778",
  receiptsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",
  sha3Uncles: "0x1dcc4de8dec75d7aab85b567b6ccd41ad312451b948a7413f0a142fd40d49347",
  size: 609,
  stateRoot: "0x4d859554d617ddf6e59474da15f2438b0485ef0ce98b7dc17405739a73258b05",
  timestamp: 1774976175,
  totalDifficulty: 711,
  transactions: [],
  transactionsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",
  uncles: []
}
```

Conclusion

Private blockchain was created using geth, the nodes in the blockchain were configured and we interacted with the blockchain by making a transaction of 10 ethers.